

PMDM11.- Web Services.

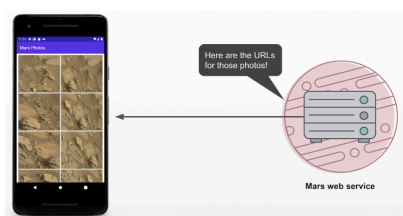


[Ministerio de Educación y Formación Profesional](https://eadfp.murciaeduca.es/pluginfile.php/117060/mod_resource/content/4/singlepage_index.html). (Dominio público)

Materiales formativos de FP Online propiedad del Ministerio de Educación y Formación Profesional.

[Aviso Legal](#) 

Caso práctico



[Google Developers](#) (Uso educativo nc)

Una de las aplicaciones que BK Programación ha desarrollado es para un cliente con presencia en distintos países del mundo. Esto implica que además del factor multidioma hay que tener en cuenta otra cuestión importante como la monedas - divisas- utilizadas en su tienda.

- Tenemos un cliente con tienda en distintos países del mundo -explica Ada a su equipo-, y necesita que se pueda comprar en su tienda en la divisa que corresponda a cada país.

- Eso es un problema -razona Juan-, puesto que el valor de la divisa fluctúa cada día. Para saber cuánto cuesta un producto de nuestro cliente en distintas monedas, necesitaríamos dedicar varios minutos cada día (sin excepción) para calcular, por ejemplo, cuántos dólares (USD) cuestan los productos fabricados en euros (EUR). Pero seguro que ya tienes una solución al problema y nos la vas a mostrar.

- ¡Por supuesto! -exclama Ada-. No se puede dedicar tiempo cada día a realizar la conversión, debe ser algo automático. Por ello es necesario que nuestra aplicación se conecte con algún servicio que ofrezca esta información publicada en Internet. El mecanismo normal más extendido actualmente es publicar esta información siguiendo unos estándares de llamadas que usan el protocolo HTTP y responden la información usando formatos como el XML o el JSON: estamos hablando de REST.

- ¿Y de este modo se puede acceder sólo a texto publicado? -pregunta María.

- No -responde Ada-. Además de formato texto, es posible también publicar la URL de distintas imágenes.

En la figura se ve cómo la NASA, por ejemplo, publica imágenes capturadas por sus robots en Marte.

El equipo se dispone a trabajar con distintas librerías como Retrofit que, con conocimientos de programación orientada a objetos (Clases, POJO, etc.), facilitarán la labor de hacer aplicaciones Android más y mejor conectadas.

1.- HttpURLConnection.

HttpURLConnection es una clase específica que a través del protocolo HTTP permite a los sistemas clientes (habitualmente navegadores, aunque en nuestro caso, el propio dispositivo Android será el cliente sin tener que mediar un navegador para acceder al contenido) acceder al contenido de páginas web.

Los navegadores hacen uso de la primitiva o método **GET** de HTTP para descargarse una página web del servidor. Es decir, cuando en un navegador web cargamos la URL: `https://dominio.com/dir/pagina.html`, internamente se está haciendo un **GET** `dominio.com/dir/pagina.html`. El servidor recibe esta petición y envía al cliente la página en formato HTML.

De la misma forma, nuestro dispositivo Android, con las librerías adecuadas, será capaz de reproducir directamente el proceso anterior. Dichas librerías principalmente son de dos grupos, las librerías **java.net.*** y las librerías **org.apache.commons.httpclient.***.

Veremos un ejemplo de uso de HttpURLConnection, que pertenece al primer grupo de las librerías anteriores. Concretamente, usaremos las clases:

- ✔ `java.net.HttpURLConnection`
- ✔ `java.net.URL`
- ✔ `java.net.URLEncoder`

1.1.- Ejercicio (URLConnection).

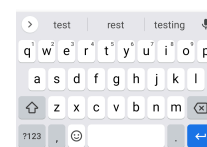
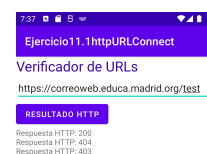
resuelto

1

Ejercicio Resuelto

Realizaremos una aplicación que solicitará al usuario una URL (incluyendo el protocolo) y se conectará a ella para devolver el resultado HTTP de la misma. Te mostramos en la imagen cómo quedaría la aplicación con estos ejemplos:

- ✓ `https://www.educa2.madrid.org/educamadrid/"` OK 200
- ✓ `https://www.educa2.madrid.org/educamadridNO/"` No encontrado 404
- ✓ `https://correoweb.educa.madrid.org/test` Forbidden 403



[Google Developers](#) (Uso educativo nc)

Es importante que conozcas los distintos [códigos de respuesta del protocolo HTTP](#) .

NOTA: Debes tener en cuenta, que en ocasiones, dependiendo de la calidad del certificado del servidor al que te conectes puedes recibir códigos de estado no esperados.

Mostrar retroalimentación

AndroidManifest.xml

Lo primero será dar permiso de uso de Internet a través del **AndroidManifest.xml** mediante la directiva:

```
1 | <uses-permission android:name="android.permission.INTERNET"/>
```

activity_main.xml

Además crearemos una interfaz básica a través de **activity_main.xml**:

```

1  <?xml version="1.0" encoding="utf-8"?>
2  <LinearLayout xmlns:android="http://schemas.android.com/apk/res/a
3      xmlns:app="http://schemas.android.com/apk/res-auto"
4      xmlns:tools="http://schemas.android.com/tools"
5      android:layout_width="match_parent"
6      android:layout_height="match_parent"
7      android:orientation="vertical"
8      tools:context="com.cidead.pmdm.ejercicio111httpurlconnect.Mai
9
10     <TextView
11         android:layout_width="wrap_content"
12         android:layout_height="wrap_content"
13         android:text="Verificador de URLs"
14         android:textSize="25dp"
15         android:textColor="@color/purple_700"
16         android:textAlignment="center"/>
17
18     <EditText android:id="@+id/et1"
19         android:layout_height="wrap_content"
20         android:layout_width="match_parent"
21         android:hint="URL a verificar"
22         android:text="https://www.educa2.madrid.org/educamadrid/"
23
24     <Button android:id="@+id/btn1"
25         android:layout_height="wrap_content"
26         android:layout_width="wrap_content"
27         android:onClick="buscar"
28         android:text="Resultado HTTP"/>
29
30     <TextView android:id="@+id/tv1"
31         android:layout_height="match_parent"
32         android:layout_width="match_parent"/>
33
34 </LinearLayout>

```

MainActivity.java

Por último, crearemos el **MainActivity.java**:

```

package com.cidead.pmdm.ejercicio111httpurlconnect;

import androidx.appcompat.app.AppCompatActivity;

import android.os.Bundle;
import android.os.StrictMode;
import android.util.Log;

```

```

8  import android.view.View;
9  import android.widget.EditText;
10 import android.widget.TextView;
11 import java.net.HttpURLConnection;
12 import java.net.URL;
13
14 public class MainActivity extends AppCompatActivity {
15     private EditText et1;
16     private TextView tv1;
17
18     @Override
19
20     protected void onCreate(Bundle savedInstanceState) {
21         super.onCreate(savedInstanceState);
22         setContentView(R.layout.activity_main);
23         et1 = (EditText) findViewById(R.id.et1);
24         tv1 = (TextView) findViewById(R.id.tv1);
25         //Con la siguiente línea permitiremos que se pueda accede
26         StrictMode.setThreadPolicy(new StrictMode.ThreadPolicy.Bu
27     }
28
29     public void buscar(View view) {
30         try {
31             String urlBusqueda = et1.getText().toString();
32             String resultado = resultadosHTTP(urlBusqueda);
33             tv1.append("Respuesta HTTP: " + resultado + "\n");
34         } catch (Exception e) {
35             tv1.append("Error al conectar\n");
36             Log.e("HTTP", e.getMessage(), e);
37         }
38     }
39
40
41     String resultadosHTTP(String urlABuscar) throws Exception {
42         String resultado = "No conectado";
43         try {
44             //URL url = new URL("https://google.es", "UTF-8") + "
45             URL url = new URL(urlABuscar);
46             //Preparamos la conexión con la clase HttpURLConnection
47             HttpURLConnection conexion = (HttpURLConnection) url.
48             //Establecemos cabeceras HTTP. Podemos asignar a la c
49             //"Mozilla/5.0 (Windows NT 6.1)", pero si lo dekjamos
50             conexion.setRequestProperty("User-Agent", "");
51             //Con el método getResposeCode() estableceremos la co
52             resultado = String.valueOf(conexion.getResponseCode())
53         } catch (Exception e) {
54             tv1.append("Error en la URL\n");
55             Log.e("HTTP", e.getMessage(), e);
56         }
57         return resultado;
58     }
59 }

```

2.- REST.

De la Wikipedia:

La **transferencia de estado representacional** (en inglés representational state transfer) o **ReST** es un estilo de arquitectura software para sistemas hipermedia distribuidos como la World Wide Web. El término se originó en el año 2000, en una tesis doctoral sobre la web escrita por Roy Fielding, uno de los principales autores de la especificación del protocolo HTTP, y ha pasado a ser ampliamente utilizado por la comunidad de desarrollo.

REST describe cualquier interfaz entre sistemas que utilice directamente HTTP para obtener datos o indicar la ejecución de operaciones sobre los datos en cualquier formato (XML, JSON, etc.), sin las abstracciones adicionales de los protocolos basados en patrones de intercambio de mensajes, como por ejemplo SOAP.

REST afirma que la web ha disfrutado de escalabilidad como resultado de una serie de diseños fundamentales clave:

- ✓ Un protocolo **cliente/servidor sin estado**: cada mensaje HTTP contiene toda la información necesaria para comprender la petición. Como resultado, ni el cliente ni el servidor necesitan recordar ningún estado de las comunicaciones entre mensajes. Sin embargo, en la práctica, muchas aplicaciones basadas en HTTP utilizan cookies y otros mecanismos para mantener el estado de la sesión (algunas de estas prácticas, como la reescritura de URLs, no son permitidas por REST)
- ✓ Un conjunto de **operaciones bien definidas** que se aplican a todos los recursos de información: HTTP en sí define un conjunto pequeño de operaciones, las más importantes son **POST**, **GET**, **PUT** y **DELETE**. Con frecuencia estas operaciones se equiparan a las operaciones CRUD en bases de datos (CLAB en castellano: crear, leer, actualizar, borrar) que se requieren para la persistencia de datos, aunque POST no encaja exactamente en este esquema.
- ✓ Una **sintaxis universal** para identificar los recursos. En un sistema REST, cada recurso es direccionable únicamente a través de su URI.
- ✓ El uso de **hipermedios**, tanto para la información de la aplicación como para las transiciones de estado de la aplicación: la representación de este estado en un sistema REST son típicamente **HTML** o **XML**. Como resultado de esto, es posible navegar de un recurso REST a muchos otros simplemente siguiendo enlaces sin requerir el uso de registros u otra infraestructura adicional

Debes conocer

Existen diversos mecanismos para consumir un servicio web. Los más popularizados son SOAP y REST. Por simplificación, en esta unidad veremos solamente REST, aunque es importante que conozcas las diferencias entre ambos mecanismos, puede que en algún momento tengas que utilizarlas. Las diferencias principales entre SOAP y REST son:

- ✓ SOAP significa Protocolo simple de acceso a objetos, mientras que REST significa Transferencia de estado representacional.

- ✓ SOAP es un protocolo mientras que REST es un patrón arquitectónico.
- ✓ SOAP usa interfaces de servicio para exponer su funcionalidad a las aplicaciones cliente, mientras que REST usa localizadores de servicios uniformes para acceder a los componentes en el dispositivo de hardware.
- ✓ SOAP necesita más ancho de banda para su uso, mientras que REST no necesita mucho ancho de banda.
- ✓ Comparando SOAP vs REST API, SOAP solo funciona con formatos XML, mientras que REST funciona con texto sin formato, XML, HTML y JSON. SOAP no puede hacer uso de REST mientras que REST puede hacer uso de SOAP.

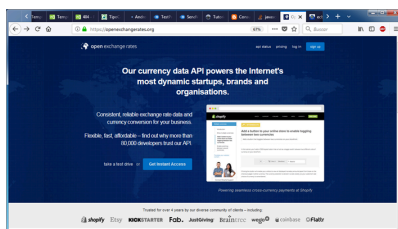
2.1.- Ejercicio resuelto 2 (Fuentes de datos REST).

Existen múltiples fuentes de datos que proporcionan servicio web mediante REST.

Una de ellas es: <https://openexchangerates.org> 

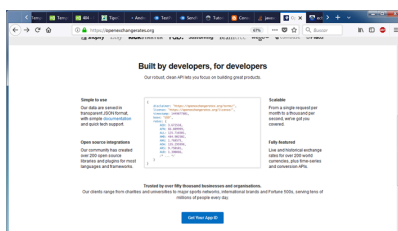
Si nos conectamos a dicha URL (figura 1), bajando en esa ventana vemos que hay un botón para obtener un ID de uso, ya que es habitual que para evitar abusos de utilización haya que registrarse (algunos otros servicios no requieren identificación ni API key) (figura 2). Utilizaremos el plan gratuito (figura 3).

Figura 1. Acceso Open Exchange Rates



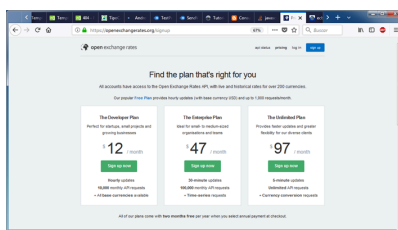
[Openexchangerates.org](https://openexchangerates.org) (Uso educativo nc)

Figura 2. Obtención de ID.



[Openexchangerates.org](https://openexchangerates.org) (Uso educativo nc)

Figura 3. Plan gratuito.

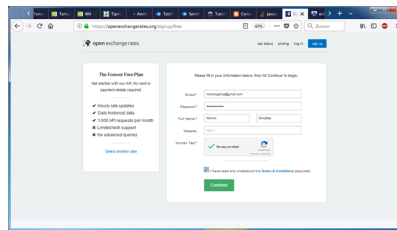


[Openexchangerates.org](https://openexchangerates.org) (Uso educativo nc)

Nos registramos (figura 4). Una vez validados, podemos entrar en nuestro escritorio (figura 5)

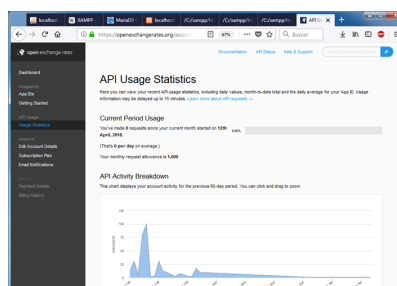
y buscar la App ID que tenemos (figura 6).

Figura 4. Registro.



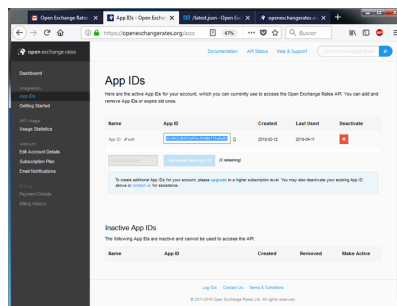
[Openexchangerates.org](https://openexchangerates.org) (Uso educativo nc)

Figura 5. Acceso a escritorio.



[Openexchangerates.org](https://openexchangerates.org) (Uso educativo nc)

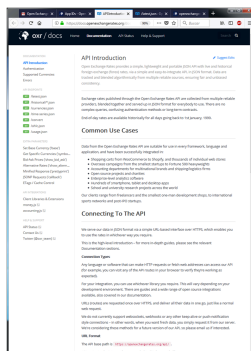
Figura 6. Búsqueda de la App ID



[Openexchangerates.org](https://openexchangerates.org) (Uso educativo nc)

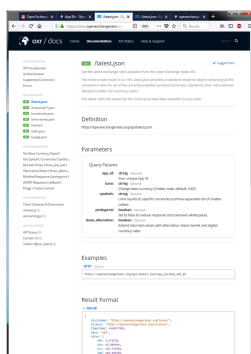
Ahora tendremos que buscar API *Endpoints*. Para ello, vamos al enlace "*Getting Started*" de la ventana anterior, abriéndose la ventana mostrada en la figura 7. De entre las API EndPoints cogemos, por ejemplo, la primera (/latest.json) (figura 8). Dicha página describe como se utiliza (definición y parámetros que admite) dicha API Endpoint.

Figura 7. Enlace "Getting Started"



[Openexchangerates.org](https://openexchangerates.org) (Uso educativo nc)

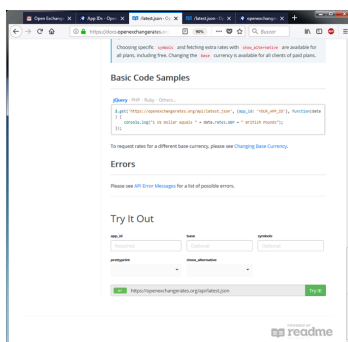
Figura 8. Seleccionamos /latest.json



[Openexchangerates.org](https://openexchangerates.org) (Uso educativo nc)

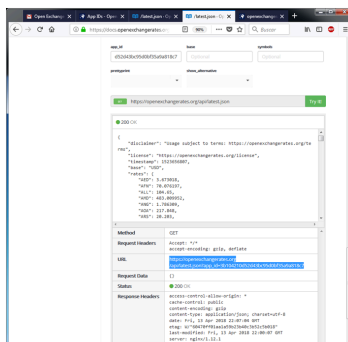
En la parte de abajo podemos probarla, para lo cual debemos suministrar obligatoriamente la App ID anteriormente mencionada (figura 9) y darle a "Try It!" (figura 10):

Figura 9. Suministrar la App ID



[Openexchangerates.org](https://openexchangerates.org) (Uso educativo nc)

Figura 10. Try it!



[Openexchangerates.org](https://openexchangerates.org) (Uso educativo nc)

Podemos ver que se genera la última valoración (ver que la fecha coincide con la del día actual en el que hacemos la petición) de cambio del dólar (base) con respecto a las demás monedas, usando formato **JSON**.

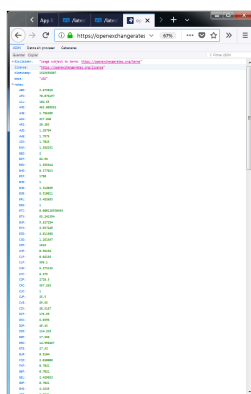
Además podemos poner la URL en un navegador e igualmente aparecerán los resultados en formato **JSON** (figura 11). Dicho fichero será descargable, pero en todo caso será preferible mantener como fuente de datos al servidor web de openexchangerates.org para asegurarnos la puntual actualización de los datos.

Si ahora intentamos cambiar la base a euros (EUR), vemos que desafortunadamente nos da un mensaje de error diciendo que el cambio de base está reservado a planes de pago, por lo cual el único cambio de base que podemos hacer es el de dólar americano (USD) a todo lo demás (figura 12).

En todo caso, este problema no es insalvable ya que será posible hacer el cambio a través de un simple algoritmo matemático, partiendo del cambio de dólar americano a todo lo demás, para poder hacer cualquier conversión posible.

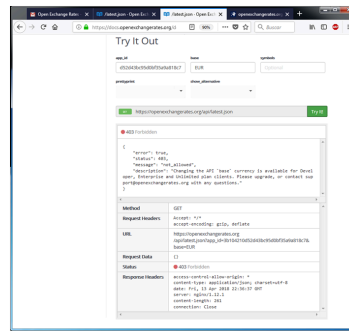
Podría igualmente llevar la URL anterior a aplicaciones como SoapUI para probar el acceso al WebService anterior (figura 13).

Figura 11. Resultados en formato JSON.

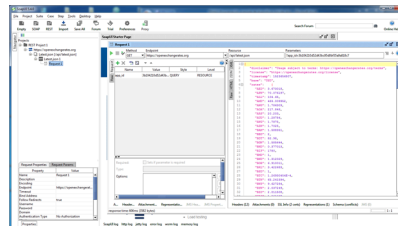


[Openexchangerates.org](https://openexchangerates.org) (Uso educativo nc)

Figura 12. Cambio de la base a euros.



[Openexchangerates.org](https://openexchangerates.org) (Uso educativo nc)
Figura 13. Uso de SoapUI.



[Openexchangerates.org](https://openexchangerates.org) (Uso educativo nc)

Como vemos, este es un ejemplo de como somos capaces de acceder a servicios REST provistos por web services externos. Hay muchos más casos, solo es cuestión de investigar en Internet. En todo caso, el consumo de estos web services es algo complicado, pero gracias a ciertas librerías que veremos más adelante, el acceso y manipulación de datos será más sencillo. Concretamente veremos Retrofit.

3.- Consumo de un Web Service.

Un web service por tanto no es más que una forma de estructurar un conjunto de acciones que permiten a las aplicaciones interactuar con un servidor para consultar datos o modificarlos mediante los métodos del protocolo HTTP.

Para desarrollar la parte servidor (el web service en sí) puede usarse cualquier tecnología capaz de publicar información mediante HTTP (PHP, ASP.NET, Java + TOMAT, etc.). Para "usar" o "consumir" este web service desarrollado con REST es necesario establecer conexiones HTTP mediante las aplicaciones. Es decir, las aplicaciones se conectan a Internet (sin usar un navegador) y son capaces de descargar e incluso subir datos al servidor.

Web service en el servidor

Para desarrollar la parte de servidor existen herramientas web sobradamente preparadas que proporcionan un servicio web completamente fiable. Estará especialmente indicado la instalación del paquete **XAMPP**, **WAMP**, o **LAMP**, dependiendo de la distribución y del sistema operativo. **XAMPP**, por ejemplo, es un paquete de software libre que proporciona una serie de aplicaciones de alto nivel, entre las cuales destaca el servidor web Apache, el sistema de gestión de bases de datos MySQL y los intérpretes para lenguajes de script PHP y Perl. El nombre proviene del acrónimo de las aplicaciones anteriormente indicadas para cualquier sistema operativo: X (se refiere a cualquiera de los diferentes sistemas operativos más utilizados), **A**pache, **M**ariaDB/MySQL, **P**HP, **P**erl.

A partir de la versión 5.6.15, **XAMPP** cambió la base de datos **MySQL** por **MariaDB**, la cual es una versión actualizada de MySQL con licencia **GPL**, y que surgió con el propósito de garantizar que los usuarios finales pudieran seguir haciendo uso de un sistema gestor de bases de datos avanzado, cosa que quedaba en entredicho por la amenaza de que **Oracle**, propietaria de MySQL pudiera comenzar a distribuir este SGBD bajo licencias no libres.

El programa proporciona un servidor web libre de altas prestaciones y muy fiable, fácil de usar y capaz de interpretar páginas dinámicas. Actualmente, XAMPP está disponible para Windows, Linux, y OS X. El XAMPP podemos instalarlo tanto en un equipo local, como en un sistema remoto.

Los certificados de servidor

Uno de los aspectos en los que más está evolucionando Android, según avanzan las APIs, es la seguridad. Esto ha hecho que desde hace tiempo las aplicaciones sólo tienen disponible conexiones a servidores HTTPS, es decir, a servidores que cuentan con un certificado que garantiza la encriptación de la comunicación. Conseguir esto en un servidor Apache es algo sencillo porque existen algunas formas de instalar lo que se denominan certificados autofirmados que permiten este cifrado en las comunicaciones. Sin embargo, las restricciones de Android desde la API 28 han aumentado, y ahora las aplicaciones no tienen habilitado el acceso a ningún servicio web que no ofrezca HTTPS con un certificado avalado por una **CA** (autoridad certificadora) que asegure que el certificado pertenece a la empresa o persona que dice ser. Esto es necesario incluso cuando accedemos a direcciones IP de nuestra propia red o incluso a nuestra máquina anfitriona desde nuestro Android Studio.

Estas nuevas medidas complican la posibilidades de realizar aplicaciones que consuman servicios web desarrollados por nosotros mismos ya que es necesario instalar certificados **reales** en el servidor o crear entidades certificadoras, lo cual excede de los objetivos del módulo.

Autoevaluación

Sobre REST, indica las afirmaciones correctas...

- ☐ Puede usarse para conseguir datos de un servicio web e incluso subir datos al servidor con los métodos HTTP.

- ☐ No guarda estado.

- ☐ Devuelve siempre la información en formato JSON.

- ☐ En un sistema REST, cada recurso es direccionable mediante su URI.

Mostrar retroalimentación

Solución

1. Correcto
2. Correcto
3. Incorrecto
4. Correcto

3.1.- Ejercicio resuelto 3 (Consumo de un Web Service).

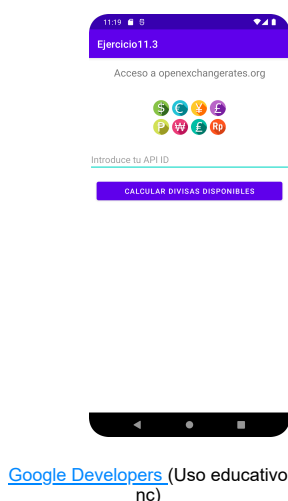
Ejercicio Resuelto

A partir del web service público con el que trabajamos en la unidad anterior (<https://openexchangerates.org/api/latest.json>) que devuelve un conjunto de divisas y los valores de cambio de las mismas con respecto al dólar (USD) vamos a realizar una aplicación capaz de hablar con este servicio para que calcule el número de divisas disponibles.

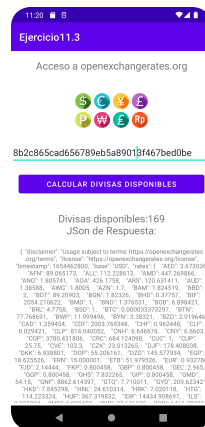
Para realizar el ejercicio, la aplicación solicitará al usuario el ID de API que debes conseguir tal como se explicó en el ejercicio anterior. En caso de que este no sea correcto, la aplicación devolverá un mensaje de error a través de TOAST. En caso de que pueda conectar, la aplicación simplemente contará las divisas que tiene disponibles y mostrará el JSON devuelto por el servicio web.

Este es el aspecto que tendrá la aplicación antes y después de establecer la conexión con el servicio.

Antes de establecer la conexión con el servicio.



Después de establecer la conexión con el servicio.



[Google Developers](#) (Uso educativo
nc)

Nota: Puedes encontrar cualquier imagen relacionada con las divisas en Internet para usarla como recurso "drawable".

Mostrar retroalimentación

AndroidManifest.xml

```
1 | <uses-permission android:name="android.permission.INTERNET"/>
```

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/a
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    tools:context="com.cidead.pdm.ejercicio113.MainActivity">

    <TextView
        android:id="@+id/tvTitulo"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:gravity="center"
        android:text="Acceso a openexchangerates.org"
        android:textSize="20dp"
        tools:text="API ID"
```

```
18         android:layout_margin="15dp"/>
19
20     <ImageView
21         android:id="@+id/imgvLogin"
22         android:layout_width="match_parent"
23         android:layout_height="80dp"
24         app:srcCompat="@drawable/divisas"
25         android:layout_margin="20dp"/>
26
27     <EditText
28         android:id="@+id/etAPIID"
29         android:layout_width="match_parent"
30         android:layout_height="wrap_content"
31         android:hint="Introduce tu API ID"/>
32
33     <Button
34         android:id="@+id/btnLogin"
35         android:layout_width="match_parent"
36         android:layout_height="wrap_content"
37         android:text="Calcular divisas disponibles"
38         android:layout_margin="15dp"/>
39
40     <TextView
41         android:id="@+id/tvDivisasEncontradas"
42         android:layout_width="match_parent"
43         android:layout_height="wrap_content"
44         android:gravity="center"
45         android:textSize="20dp"
46         android:layout_margin="15dp"/>
47
48     <TextView
49         android:id="@+id/tvJsonSent"
50         android:layout_width="match_parent"
51         android:layout_height="wrap_content"
52         android:gravity="center"
53         android:textSize="12dp"
54         android:layout_margin="5dp"/>
55
56 </LinearLayout>
```

MainActivity.java

```
package com.cidead.pdm.ejercicio113;

import androidx.appcompat.app.AppCompatActivity;

import android.content.Intent;
import android.os.Bundle;
import android.util.Log;
import android.view.View;
import android.widget.Button;
import android.widget.EditText;
```

```

import android.widget.TextView;
import android.widget.Toast;

import org.json.JSONArray;
import org.json.JSONObject;

import java.io.DataOutputStream;
import java.net.HttpURLConnection;
import java.net.URL;
import java.util.Scanner;

public class MainActivity extends AppCompatActivity {
    EditText etAPIID;
    TextView tvDivisasEncontradas, tvJsonSent;
    Button btnLogin;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        etAPIID=(EditText)findViewById(R.id.etAPIID);
        tvDivisasEncontradas =(TextView)findViewById(R.id.tvDivis
        tvJsonSent = (TextView)findViewById(R.id.tvJsonSent);
        btnLogin=(Button)findViewById(R.id.btnLogin);
        btnLogin.setOnClickListener(new View.OnClickListener() {

            @Override
            public void onClick(View view) {
                //Creamos un hilo para conectar probando la APIID
                Thread hilo=new Thread(){
                    @Override
                    public void run() {
                        super.run();
                        final String respuesta=enviarPost(etAPIID
                        runOnUiThread(new Runnable() { //El método
                            @Override
                            public void run() {
                                //Limpiamos por si se repite la a
                                tvDivisasEncontradas.setText("");
                                tvJsonSent.setText("");
                                //Toast.makeText(getApplicationContextCo
                                int numElementosEncontrados = obj
                                if (numElementosEncontrados>0){
                                    tvDivisasEncontradas.setText(
                                        "JSon de Respuesta:")
                                    tvJsonSent.setText(respuesta)
                                }
                                else{
                                    Toast.makeText(getApplicationContext
                                }
                            }
                        });
                    }
                });
                hilo.start(); //Con esto arrancamos el hilo
            }
        });
    }
}

```

```

69
70
71     public String enviarPost(String app_id){
72         String parametros="app_id="+app_id; //Los literales corr
73         HttpURLConnection conexion=null;
74         String respuesta="";
75         try{
76             //Tenemos que consultar la página enviándole los dato
77             URL url=new URL("https://openexchangerates.org/api/la
78             conexion=(HttpURLConnection)url.openConnection();
79             conexion.setRequestMethod("POST");
80             conexion.setRequestProperty("Content-Length", "");
81             conexion.setDoOutput(true);
82             DataOutputStream dos=new DataOutputStream((conexion.g
83             dos.close());
84             //Recogeremos la respuesta que nos devuelve el WebSer
85             Scanner lectura=new Scanner(conexion.getInputStream())
86             while (lectura.hasNextLine())
87                 respuesta += lectura.nextLine();
88         }catch (Exception e){
89             Log.e("Error",e.toString());
90         }
91         return respuesta.toString();
92     }
93
94     //Creamos un método para contar cuantas divisas devuelve la c
95     //Pasaremos como parámetro la respuesta que da el método envi
96     public int objetoJSON(String respuesta){
97         int numDivisas = 0;
98         try{
99             JSONObject jobject = new JSONObject(respuesta);
100             JSONObject divisas = jobject.getJSONObject("rates");
101             numDivisas = divisas.length();
102             //Log.d("Divisas", Integer.toString(numDivisas));
103         }catch (Exception e){
104             Log.e("Error", e.getMessage());
105         }
106         return numDivisas;
107     }

```

Este es el texto JSON devuelto por el servicio

```

{
  "disclaimer": "Usage subject to terms: https://openexchangerate
  "license": "https://openexchangerates.org/license",
  "timestamp": 1654365613,
  "base": "USD",
  "rates": {
    "AED": 3.6731,
    "AFN": 89.162943,
    "ALL": 112.223267,
    "AMD": 447.269866,

```

"ANG": 1.805741,
"AOA": 426.1758,
"ARS": 120.67334,
"AUD": 1.387465,
"AWG": 1.8005,
"AZN": 1.7,
"BAM": 1.824519,
"BBD": 2,
"BDT": 89.20903,
"BGN": 1.821413,
"BHD": 0.37697,
"BIF": 2060.263278,
"BMD": 1,
"BND": 1.376531,
"BOB": 6.898421,
"BRL": 4.775,
"BSD": 1,
"BTC": 0.000033644322,
"BTN": 77.768691,
"BWP": 11.999496,
"BYN": 3.38321,
"BZD": 2.019646,
"CAD": 1.25957,
"CDF": 2004.038697,
"CHF": 0.961757,
"CLF": 0.029469,
"CLP": 811.81,
"CNH": 6.657479,
"CNY": 6.6603,
"COP": 3781.3558,
"CRC": 684.124098,
"CUC": 1,
"CUP": 25.75,
"CVE": 103.3,
"CZK": 23.0091,
"DJF": 178.372548,
"DKK": 6.9387,
"DOP": 55.207702,
"DZD": 145.41,
"EGP": 18.630186,
"ERN": 15.000001,
"ETB": 52.074408,
"EUR": 0.932923,
"FJD": 2.1368,
"FKP": 0.800737,
"GBP": 0.800737,
"GEL": 2.965,
"GGP": 0.800737,
"GHS": 7.840332,
"GIP": 0.800737,
"GMD": 54.15,
"GNF": 8870.767996,
"GTQ": 7.710011,
"GYD": 209.623421,
"HKD": 7.8448,
"HNL": 24.622892,
"HRK": 7.0142,
"HTG": 114.038788,

"HUF" : 365.63,
"IDR" : 14433.5,
"ILS" : 3.335,
"IMP" : 0.800737,
"INR" : 77.6934,
"IQD" : 1462.353565,
"IRR" : 42350,
"ISK" : 129.03,
"JEP" : 0.800737,
"JMD" : 154.180666,
"JOD" : 0.7102,
"JPY" : 130.845,
"KES" : 116.978189,
"KGS" : 79.525751,
"KHR" : 4068.935217,
"KMF" : 458.749854,
"KPW" : 900,
"KRW" : 1251.53,
"KWD" : 0.306277,
"KYD" : 0.834911,
"KZT" : 435.599831,
"LAK" : 13390.901035,
"LBP" : 1514.968721,
"LKR" : 361.204622,
"LRD" : 151.999978,
"LSL" : 15.510228,
"LYD" : 4.786048,
"MAD" : 9.878728,
"MDL" : 19.067231,
"MGA" : 4055.425659,
"MKD" : 57.492275,
"MMK" : 1855.118735,
"MNT" : 3105.3554,
"MOP" : 8.095189,
"MRU" : 36.541135,
"MUR" : 43.150001,
"MVR" : 15.435,
"MWK" : 816.25,
"MXN" : 19.54515,
"MYR" : 4.391,
"MZN" : 63.850001,
"NAD" : 15.49,
"NGN" : 415.814006,
"NIO" : 35.919846,
"NOK" : 9.43645,
"NPR" : 124.429799,
"NZD" : 1.535745,
"OMR" : 0.385029,
"PAB" : 1,
"PEN" : 3.713443,
"PGK" : 3.530509,
"PHP" : 52.909995,
"PKR" : 198.136548,
"PLN" : 4.28225,
"PYG" : 6861.311273,
"QAR" : 3.641,
"RON" : 4.611,
"RSD" : 109.539103,

```
128     "RUB" : 63.360006,  
129     "RWF" : 1022.445353,  
130     "SAR" : 3.750665,  
131     "SBD" : 8.116969,  
132     "SCR" : 14.330625,  
133     "SDG" : 455.5,  
134     "SEK" : 9.7671,  
135     "SGD" : 1.3761,  
136     "SHP" : 0.800737,  
137     "SLL" : 12887.2,  
138     "SOS" : 579.587715,  
139     "SRD" : 21.22,  
140     "SSP" : 130.26,  
141     "STD" : 22994,  
142     "STN" : 23.15,  
143     "SVC" : 8.767037,  
144     "SYP" : 2512.53,  
145     "SZL" : 15.512281,  
146     "THB" : 34.29387,  
147     "TJS" : 11.306259,  
148     "TMT" : 3.51,  
149     "TND" : 3.02,  
150     "TOP" : 2.299263,  
151     "TRY" : 16.4176,  
152     "TTD" : 6.791608,  
153     "TWD" : 29.375,  
154     "TZS" : 2328,  
155     "UAH" : 29.457448,  
156     "UGX" : 3745.800949,  
157     "USD" : 1,  
158     "UYU" : 40.022165,  
159     "UZS" : 11015.948404,  
160     "VES" : 5.11435,  
161     "VND" : 23192,  
162     "VUV" : 114.629832,  
163     "WST" : 2.612952,  
164     "XAF" : 611.957277,  
165     "XAG" : 0.04559972,  
166     "XAU" : 0.00054022,  
167     "XCD" : 2.70255,  
168     "XDR" : 0.727495,  
169     "XOF" : 611.957277,  
170     "XPD" : 0.00050545,  
171     "XPF" : 111.32731,  
172     "XPT" : 0.00098188,  
173     "YER" : 250.249937,  
174     "ZAR" : 15.543525,  
175     "ZMW" : 17.033219,  
176     "ZWL" : 322  
177 }
```

4.- Retrofit.

Retrofit es una tecnología basada en REST con la cual podremos desarrollar aplicaciones en Android que accedan a servicios web de una forma relativamente sencilla.

Para usar esta tecnología debemos incorporar dos librerías:

- ✓ La librería Retrofit para Android y Java (compatible con Kotlin) para hacer llamadas de red y obtener el resultado.
- ✓ Una librería que nos permita convertir el resultado devuelto “_____parseando” de forma automática a su objeto; esto facilita mucho realizar peticiones a un API y procesar la respuesta.

Dispone de métodos que manejan API REST con contenidos en formato XML o en formato JSON. Retrofit puede manejar las clásicas primitivas de HTTP (GET, POST, PUT, HEAD...).

El resultado de la consulta al servicio web se incorpora en nuestra aplicación en una estructura de clases que representen los objetos de la respuesta esperada en formato POJO. Estas clases las debemos crear nosotros mismos con la misma estructura que tenga el resultado de la respuesta. Por ejemplo, si la respuesta es en formato JSON, deberemos construir una estructura de clases que represente los campos que nos interesan en la consulta de este JSON.

En este enlace puedes aprender más sobre [el cliente HTTP de Retrofit](#) .

En este otro enlace puedes ver un ejemplo en Kotlin y algunas explicaciones de [cómo obtener datos de Internet desde un cliente Retrofit en una aplicación de Android](#) .

4.1.- Ejercicio resuelto 4 (Retrofit).

Ejercicio Resuelto



[Google Developers](#) (Uso educativo nc)

Para facilitar la tarea, vamos a tratar de realizar con la tecnología Retrofit un acceso al servicio web de openexchangerates.org del ejercicio visto en el apartado de "Consumo de un Web Service". El ejercicio devolverá el cambio de seis divisas en el momento actual (puedes elegir las que tú quieras) con respecto al dólar, ya que es la información que devuelve el Web Service.

Concretamente, el aspecto que buscaremos para la salida será similar al mostrado en la captura de pantalla de móvil.

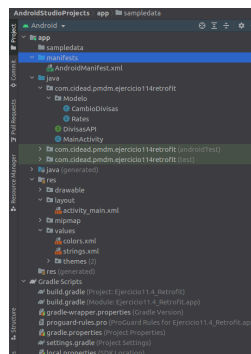
En el siguiente vídeo puedes encontrar un ejemplo de cómo construir este tipo de aplicaciones paso a paso con Android Studio.

https://www.youtube.com/embed/Lx1e_fdnNxA

[Descripción textual del vídeo Ejemplo de Retrofit](#) 

Mostrar retroalimentación

La estructura final de archivos implicados en el proyecto podría ser la siguiente:



[Google Developers](#) (Uso educativo nc)

build.gradle (Module app)

En primer lugar, incorporaremos en **build.gradle (Module app)** las dependencias:

```
1 | dependencies {  
2 |     ...  
3 |     // Retrofit  
4 |     implementation "com.squareup.retrofit2:retrofit:2.9.0"  
5 |     // Retrofit with Convertes gson  
6 |     implementation "com.squareup.retrofit2:converter-gson:2.9.0"  
7 | }
```

AndroidManifest.xml

No debemos olvidar configurar el permiso de acceso a Internet en el fichero **AndroidManifest.xml**:

```
1 | <uses-permission android:name="android.permission.INTERNET"/>
```

activity_main.xml

La interfaz es sencilla y quedará definida en el archivo **activity_main.xml**:

```
<?xml version="1.0" encoding="utf-8"?>  
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/a  
    xmlns:app="http://schemas.android.com/apk/res-auto"  
    xmlns:tools="http://schemas.android.com/tools"  
    android:layout_width="match_parent"  
    android:layout_height="match_parent"  
    android:orientation="vertical"  
    tools:context="com.cidead.pmdm.ejercicio114retrofit.MainActiv  
  
    <TextView  
        android:id="@+id/tvTitulo"  
        android:layout_width="match_parent"  
        android:layout_height="wrap_content"  
        android:gravity="center"  
        android:text="Acceso a openexchangerates.org"
```

```

16         android:textSize="20sp"
17         tools:text="API ID"
18         android:layout_margin="15dp"/>
19
20     <ImageView
21         android:id="@+id/imgvLogin"
22         android:layout_width="match_parent"
23         android:layout_height="80dp"
24         app:srcCompat="@drawable/divisas"
25         android:layout_margin="20sp"/>
26
27     <EditText
28         android:id="@+id/etAPIID"
29         android:layout_width="match_parent"
30         android:layout_height="wrap_content"
31         android:hint="Introduce tu API ID"/>
32
33     <Button
34         android:id="@+id/btnGetData"
35         android:layout_width="match_parent"
36         android:layout_height="wrap_content"
37         android:text="Obtener cambio divisas"
38         android:layout_margin="15sp"/>
39
40     <TextView
41         android:id="@+id/tvBase"
42         android:layout_width="match_parent"
43         android:layout_height="wrap_content"
44         android:gravity="center"
45         android:textSize="20sp"
46         android:layout_margin="15dp"
47     />
48
49     <TextView
50         android:id="@+id/tvConvert"
51         android:layout_width="match_parent"
52         android:layout_height="wrap_content"
53         android:inputType="textMultiLine"
54         android:gravity="center"
55         android:textSize="20sp"
56         android:layout_margin="15dp"
57     />
58
59 </LinearLayout>

```

Por otro lado, dentro de la carpeta modelos hemos creado una serie de clases auxiliares que representan el formato de los objetos que vamos a recibir del web service en formato JSON. El formato esperado en el momento de redactar este ejercicio es el mostrado a continuación. Si hubiera cambios tendrás que adaptar los nombres de los atributos de las clases usadas e incluso la estructura de los archivos para que sea similar a la estructura del JSON recibido.

Formato esperado

```
1  {
2    "disclaimer": "Usage subject to terms: https://openexchangerate
3    "license": "https://openexchangerates.org/license",
4    "timestamp": 1655024402,
5    "base": "USD",
6    "rates": {
7      "AED": 3.6731,
8      "AFN": 89.399125,
9      "ALL": 114.229019,
10     "AMD": 429.910968,
11     "ANG": 1.815174,
12     "AOA": 435.8264,
13     "ARS": 122.446573,
14     "AUD": 1.411943,
15     "AWG": 1.8005,
16     "AZN": 1.7,
17     "BAM": 1.859508,
18     "BBD": 2,
19     "BDT": 93.651536,
20     "BGN": 1.85946,
21     "BHD": 0.376878,
22     "BIF": 2071.295103,
23     "BMD": 1,
24     "BND": 1.391139,
25     ...
26     "ZWL": 322
27   }
28 }
```

CambioDivisas.java

Clase CambioDivisas.java:

```
1  package com.cidead.pmdm.ejercicio114retrofit.Modelo;
2
3  import com.google.gson.annotations.Expose;
4  import com.google.gson.annotations.SerializedName;
5
6  public class CambioDivisas {
7      @SerializedName("timestamp")
8      @Expose
9      private String timestamp;
10     @SerializedName("base")
11     private String base;
```

```

11  @Expose
12  private String base;
13  @SerializedName("rates")
14  @Expose
15  private Rates rates;
16
17  public String getTimestamp() {
18      return timestamp;
19  }
20
21  public void setTimestamp(String timestamp) {
22      this.timestamp = timestamp;
23  }
24
25  public String getBase() {
26      return base;
27  }
28
29  public void setBase(String base) {
30      this.base = base;
31  }
32
33  public Rates getRates() {
34      return rates;
35  }
36
37  public void setRates(Rates rates) {
38      this.rates = rates;
39  }
40
41  @Override
42  public String toString() {
43      return "CambioDivisas{" +
44          "timestamp='" + timestamp + '\'' +
45          ", base='" + base + '\'' +
46          ", rate=" + rates +
47          '}';
48  }
49  }

```

Rates.java

Clase Rates.java:

```

1  package com.cidead.pmdm.ejercicio114retrofit.Modelo;
2
3  import com.google.gson.annotations.Expose;
4  import com.google.gson.annotations.SerializedName;
5
6  public class Rates {
7      @SerializedName("EUR")
8      @Expose
9

```

```
10     private String EUR;
11
12     @SerializedName("AUD")
13     @Expose
14     private String AUD;
15
16     @SerializedName("CNY")
17     @Expose
18     private String CNY;
19
20     @SerializedName("GBP")
21     @Expose
22     private String GBP;
23
24     @SerializedName("ISK")
25     @Expose
26     private String ISK;
27
28     @SerializedName("MXN")
29     @Expose
30     private String MXN;
31
32     @SerializedName("PLN")
33     @Expose
34     private String PLN;
35
36     public String getEUR() {
37         return EUR;
38     }
39
40     public void setEUR(String EUR) {
41         this.EUR = EUR;
42     }
43
44     public String getAUD() {
45         return AUD;
46     }
47
48     public void setAUD(String AUD) {
49         this.AUD = AUD;
50     }
51
52     public String getCNY() {
53         return CNY;
54     }
55
56     public void setCNY(String CNY) {
57         this.CNY = CNY;
58     }
59
60     public String getGBP() {
61         return GBP;
62     }
63
64     public void setGBP(String GBP) {
65         this.GBP = GBP;
66     }
67
```

```

68     public String getISK() {
69         return ISK;
70     }
71
72     public void setISK(String ISK) {
73         this.ISK = ISK;
74     }
75
76     public String getMXN() {
77         return MXN;
78     }
79
80     public void setMXN(String MXN) {
81         this.MXN = MXN;
82     }
83
84     public String getPLN() {
85         return PLN;
86     }
87
88     public void setPLN(String PLN) {
89         this.PLN = PLN;
90     }
91
92     @Override
93     public String toString() {
94         return "Rates{" +
95             "EUR='" + EUR + '\'' +
96             ", AUD='" + AUD + '\'' +
97             ", CNY='" + CNY + '\'' +
98             ", GBP='" + GBP + '\'' +
99             ", ISK='" + ISK + '\'' +
100            ", MXN='" + MXN + '\'' +
101            ", PLN='" + PLN + '\'' +
102            '}';
103     }
}

```

Configuración de la conexión al servidor web

Además, para configurar la conexión al servidor web, tenemos la interfaz que conecta con el servicio web que mediante el método GET solicita la aplicación llamada latest.json pasando el parámetro app_id. Si quisiéramos consumir este servicio desde un navegador, tendríamos que usar esta URL con tu ID de API https://openexchangerates.org/api/latest.json?app_id=8b2c865cad6exxxx..., pero con Retrofit debemos construirla de la siguiente manera:

```

package com.cidead.pmdm.ejercicio114retrofit;

import com.cidead.pmdm.ejercicio114retrofit.Modelo.CambioDivisas;

```

```
5
6 import retrofit2.Call;
7 import retrofit2.http.GET;
8 import retrofit2.http.Headers;
9 import retrofit2.http.Query;
10
11 public interface DivisasAPI {
12     String BASE_URL = "https://openexchangerates.org/api/";
13     @Headers("Content-type: application/json")
14     @GET("latest.json")
15     Call<CambioDivisas> getStuff(@Query("app_id") String key);
16 }
```

MainActivity.java

Por último la clase MainActivity.java:

```
package com.cidead.pmdm.ejercicio114retrofit;

import androidx.appcompat.app.AppCompatActivity;

import android.os.Bundle;
import android.util.Log;
import android.view.View;
import android.widget.Button;
import android.widget.EditText;
import android.widget.TextView;
import android.widget.Toast;

import com.cidead.pmdm.ejercicio114retrofit.Modelo.CambioDivisas;
import com.cidead.pmdm.ejercicio114retrofit.Modelo.Rates;

import retrofit2.Call;
import retrofit2.Callback;
import retrofit2.Response;
import retrofit2.Retrofit;
import retrofit2.converter.gson.GsonConverterFactory;

public class MainActivity extends AppCompatActivity {
    private static final String TAG = "MainActivity";
    private static String BASE_URL = "https://openexchangerates.o

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        Button btnGetData = (Button) findViewById(R.id.btnGetData)
        EditText etAPIID = (EditText) findViewById(R.id.etAPIID);
        TextView tvBase = (TextView) findViewById(R.id.tvBase);
        TextView tvConvert = (TextView) findViewById(R.id.tvConve

        btnGetData.setOnClickListener(new View.OnClickListener()
```



```
37
38     @Override
39     public void onClick(View view) {
40         Retrofit retrofit = new Retrofit.Builder()
41             .baseUrl(DivisasAPI.BASE_URL)
42             .addConverterFactory(GsonConverterFactory)
43             .build();
44
45         DivisasAPI divisasAPI = retrofit.create(DivisasAPI.class);
46         Call<CambioDivisas> call = divisasAPI.getStuff(et
47
48         call.enqueue(new Callback<CambioDivisas>() {
49
50             @Override
51             public void onResponse
52                 (Call<CambioDivisas> call, Response<C
53
54                 tvBase.setText("Referencia 1" + response.
55                 tvConvert.setText("EUR : " + response.bod
56                     "AUD : " + respon
57                     "CNY : " + respon
58                     "ISK : " + respon
59                     "MXN : " + respon
60                     "PLN : " + respon
61
62                 //Trazas
63                 //Log.d(TAG, "onResponse: Respuesta del s
64                 //Log.d(TAG, "onResponse: Info recibida;
65             }
66
67             @Override
68             public void onFailure(Call<CambioDivisas> cal
69                 //Log.e(TAG, "Error: " + "onFailure: Algo
70                 Toast.makeText(MainActivity.this, "Algo s
71             }
72         });
73     });
74 }
```

5.- Obtención de imágenes a través de la web.

Existen varias librerías que permiten descargarnos imágenes procedentes de servidores web. Las más conocidas son Picasso, Glide, Universal Image Loader y Fresco. Además, incluso Retrofit dispone de métodos relacionados con la carga de imágenes como `getImage()`.

La librería Picasso se caracteriza por su sencillez y poco código necesario para poder cargar imágenes con buena calidad (por ejemplo la calidad de imagen que maneja Glide es algo peor, pero a diferencia de Picasso, Glide soporta formato GIF animados). Será necesario incluir la librería a través de la inclusión en el apartado de dependencias del `build.gradle` (Module: app) como hemos hecho con otras librerías, en el apartado de las dependencias.

Para saber más

Aquí puedes encontrar más detalles sobre la [librería Picasso](#)  .

5.1.- Ejercicio resuelto 5 (Imágenes con Picasso).

Ejercicio Resuelto

Se pide, usando la librería Picasso, conectar con una API de la NASA que publica imágenes de Marte capturadas por el Rover Curiosity.

NOTA: Para acceder a esta imagen no es necesario usar una API_key aunque puedes conseguirla registrándote con tu correo electrónico.

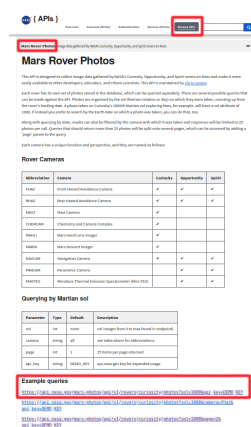
Las APIs están publicadas en esta página web: <https://api.nasa.gov/> .

Como puedes ver en la figura 1, debes ir a la página, pulsar el botón de la parte superior de "Browse APIs" y, a continuación, desplazarnos hasta abajo y encontrar los detalles de la API "Mars Rover Photos".

En la figura 2 vemos que pulsando al primer enlace (https://api.nasa.gov/mars-photos/api/v1/rovers/curiosity/photos?sol=1000&api_key=DEMO_KEY ) de "Example Queries" podemos ver la estructura del JSON devuelto por el servicio.

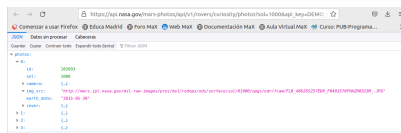
Desarrolla una App Android que muestre, mediante la librería Picasso, tres de estas imágenes servidas desde estas páginas. Para ello puedes conseguir los enlaces mostrados en alguna de las imágenes publicadas del JSON. El aspecto de la aplicación debe ser el mostrado en la figura 3.

Figura 1



[Google Developers](https://developers.google.com/) (Uso educativo nc)

Figura 2



[Google Developers](#) (Uso educativo nc)

Figura 3



[Google Developers](#) (Uso educativo nc)

Mostrar retroalimentación

AndroidManifest.xml

Añadimos el permiso al **AndroidManifest.xml**:

```
1 | <uses-permission android:name="android.permission.INTERNET"/>
```

build.gradle(:app)

En el archivo build.gradle(:app) añadimos la librería Picasso en el apartado de las dependencias:

```
1 | dependencies {
2 |
3 |     implementation 'androidx.appcompat:appcompat:1.4.2'
4 |     ...
5 |     //Añadimos la librería Picasso
6 |
7 | }
```

```

    implementation 'com.squareup.picasso:picasso:2.8'
}

```

activity_main.xml

La interfaz vendrá dada por **activity_main.xml**:

```

<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout xmlns:android=
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

    <ImageView
        android:id="@+id/iv1"
        android:layout_width="162dp"
        android:layout_height="125dp"
        tools:layout_constraintTop_creator="1"
        android:layout_marginStart="8dp"
        android:layout_marginTop="8dp"
        tools:layout_constraintLeft_creator="1"
        app:layout_constraintLeft_toLeftOf="parent"
        app:layout_constraintTop_toTopOf="parent"
        android:layout_marginLeft="8dp" />

    <TextView
        android:id="@+id/tv1"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Nasa - Curiosity - Img0"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintHorizontal_bias="0.725"
        app:layout_constraintLeft_toLeftOf="parent"
        app:layout_constraintRight_toRightOf="parent"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintVertical_bias="0.081" />

    <ImageView
        android:id="@+id/iv2"
        android:layout_width="162dp"
        android:layout_height="125dp"
        android:layout_marginLeft="8dp"
        android:layout_marginStart="8dp"
        android:layout_marginTop="180dp"
        app:layout_constraintLeft_toLeftOf="parent"
        app:layout_constraintTop_toTopOf="parent"
        tools:layout_constraintLeft_creator="1"
        tools:layout_constraintTop_creator="1" />

    <TextView

```

```

47         android:id="@+id/tv2"
48         android:layout_width="wrap_content"
49         android:layout_height="wrap_content"
50         android:text="Nasa - Curiosity - Img10"
51         app:layout_constraintBottom_toBottomOf="parent"
52         app:layout_constraintHorizontal_bias="0.748"
53         app:layout_constraintLeft_toLeftOf="parent"
54         app:layout_constraintRight_toRightOf="parent"
55         app:layout_constraintTop_toTopOf="parent"
56         app:layout_constraintVertical_bias="0.324" />
57
58     <ImageView
59         android:id="@+id/iv3"
60         android:layout_width="162dp"
61         android:layout_height="125dp"
62         android:layout_marginLeft="8dp"
63         android:layout_marginStart="8dp"
64         android:layout_marginTop="337dp"
65         app:layout_constraintLeft_toLeftOf="parent"
66         app:layout_constraintTop_toTopOf="parent"
67         tools:layout_constraintLeft_creator="1"
68         tools:layout_constraintTop_creator="1" />
69
70     <TextView
71         android:id="@+id/tv3"
72         android:layout_width="wrap_content"
73         android:layout_height="wrap_content"
74         android:text="Nasa - Curiosity - Img20"
75         app:layout_constraintBottom_toBottomOf="parent"
76         app:layout_constraintHorizontal_bias="0.748"
77         app:layout_constraintLeft_toLeftOf="parent"
78         app:layout_constraintRight_toRightOf="parent"
79         app:layout_constraintTop_toTopOf="parent"
80         app:layout_constraintVertical_bias="0.561" />
81
82 </androidx.constraintlayout.widget.ConstraintLayout>

```

MainActivity.java

Finalmente, el archivo **MainActivity.java**:

```

package com.cidead.pmdm.ejercicio115;

import androidx.appcompat.app.AppCompatActivity;

import android.os.Bundle;
import android.widget.ImageView;

import com.squareup.picasso.Picasso;

public class MainActivity extends AppCompatActivity {
    ImageView iv1;

```

```
13     ImageView iv2;
14     ImageView iv3;
15
16     @Override
17     protected void onCreate(Bundle savedInstanceState) {
18         super.onCreate(savedInstanceState);
19         setContentView(R.layout.activity_main);
20         iv1=(ImageView)findViewById((R.id.iv1));
21         iv2=(ImageView)findViewById((R.id.iv2));
22         iv3=(ImageView)findViewById((R.id.iv3));
23         //URL de la API https://api.nasa.gov/mars-photos/api/v1/r
24
25         Picasso.get()
26             .load("https://mars.nasa.gov/msl-raw-images/proj/
27             .resize(500, 500)
28             .centerCrop()
29             .into(iv1);
30         Picasso.get()
31             .load("https://mars.nasa.gov/msl-raw-images/msss/
32             .resize(500, 500)
33             .centerCrop()
34             .into(iv2);
35         Picasso.get()
36             .load("https://mars.nasa.gov/msl-raw-images/msss/
37             .resize(500, 500)
38             .centerCrop()
39             .into(iv3);
40         /* Otra forma de llamar aun mas corta
41         Picasso.get().load("https://mars.nasa.gov/msl-raw-images/
42     }
43 }
```